



Aplicaciones web 1

Actividad 2

Actividad a desarrollar en grupo. La entrega es individual para que quede el registro.

MPA/SPA y Wireframes

Continuar con el caso de negocio de la actividad 1.

Consignas

- **Mapa del sitio (Sitemap):** Diseñar la estructura jerárquica de páginas y módulos de la aplicación.
- **Definición de estrategia MPA vs. SPA:**
 - Indicar qué páginas funcionarán bajo el esquema **MPA** (Multi-Page Application) y justificar la decisión (ej. SEO, estructura independiente).
 - Indicar qué módulos funcionarán como **SPA** (Single-Page Application) y explicitar qué datos o componentes se actualizarán dinámicamente con JavaScript.
- **Bocetos a mano:** Dibujar a mano alzada en papel las 3 vistas o secciones principales de la web, tanto para **móvil** como para **escritorio**.
- **Wireframe digital:** Crear un lienzo en [Canva](#) y pasar los bocetos de papel a un wireframe de media fidelidad (Mid-Fi) utilizando grillas responsive estándar (Desktop: 1440px / Mobile: 375px).

Uso obligatorio de inteligencia artificial (IA)

- **Asistencia en Sitemap y MPA/SPA:** Utilizar la IA para solicitar una propuesta de árbol de navegación y evaluar qué partes del proyecto conviene desarrollar como SPA.

Se debe adjuntar en la entrega la captura o transcripción del prompt utilizado y la respuesta obtenida (historial).

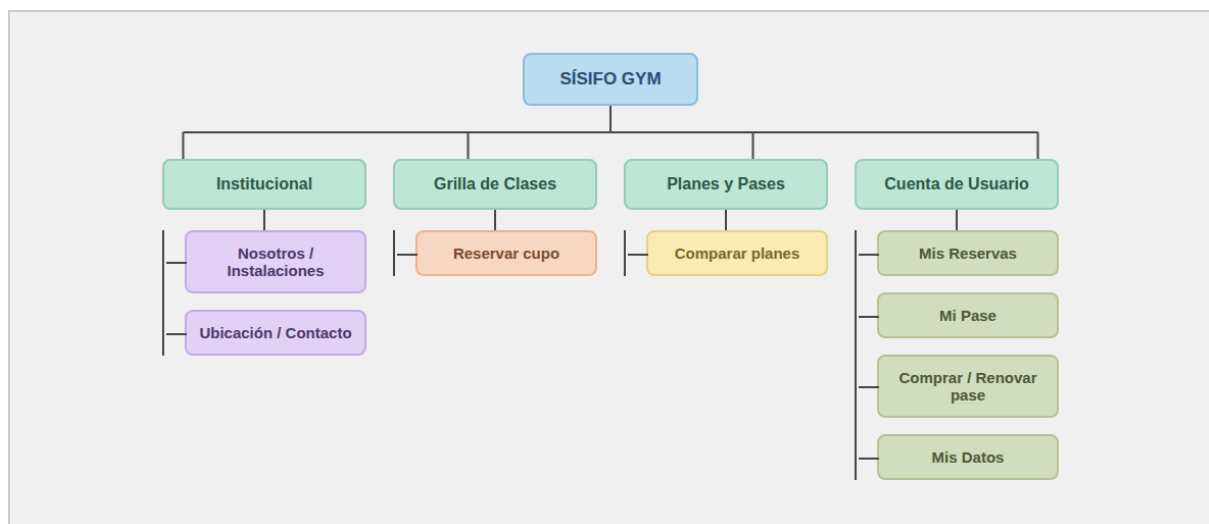
Prompt base:

"Actúa como un arquitecto de software web. Tengo un proyecto de [negocio]. Propón un sitemap en formato de lista para el sitio web y recomiéndame qué secciones deberían ser MPA y cuáles SPA, justificando el impacto en la experiencia de usuario y el rendimiento."

- **Asistencia en diseño/prompts de layout (plantilla):** Consultar a la IA sobre la distribución recomendada de componentes UX para pantallas móviles (mobile-first) según las historias de usuario definidas.
- **Tarea del grupo:** Auditar la respuesta de la IA, ajustar los textos según el criterio propio del grupo y refinar la redacción final.

Respuestas

Mapa del sitio (Sitemap):



Definición de estrategia MPA vs. SPA:

1. Institucional → MPA

Incluye: Nosotros/Instalaciones, Ubicación/Contacto.

Justificación: es contenido estático que casi no cambia, no depende de que el usuario esté logueado, y necesita ser indexado por buscadores.

2. Grilla de Clases → SPA

Incluye: la grilla semanal de clases y la acción de Reservar cupo.

Qué se actualizará: la cantidad de cupos disponibles por clase, y el resultado de la reserva en sí.

3. Planes y Pases → MPA

Incluye: la página pública donde se comparan los planes y precios.

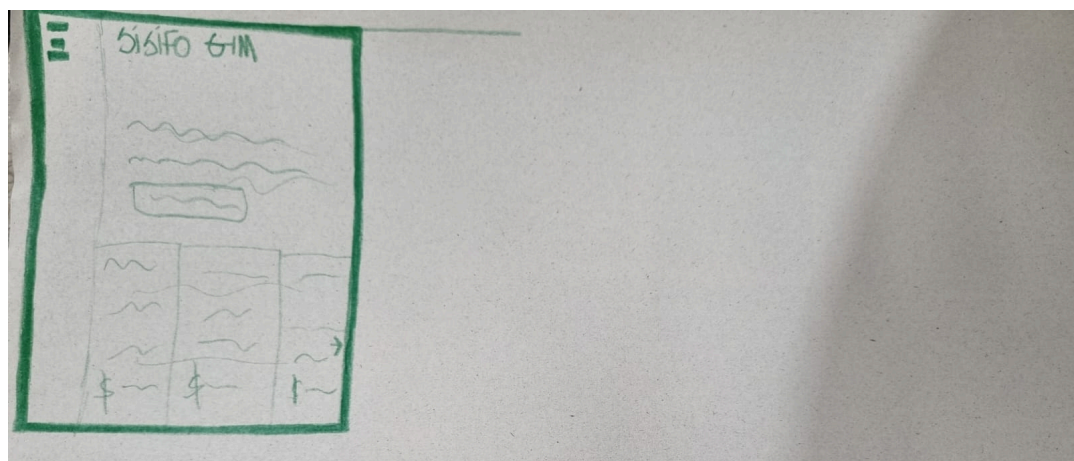
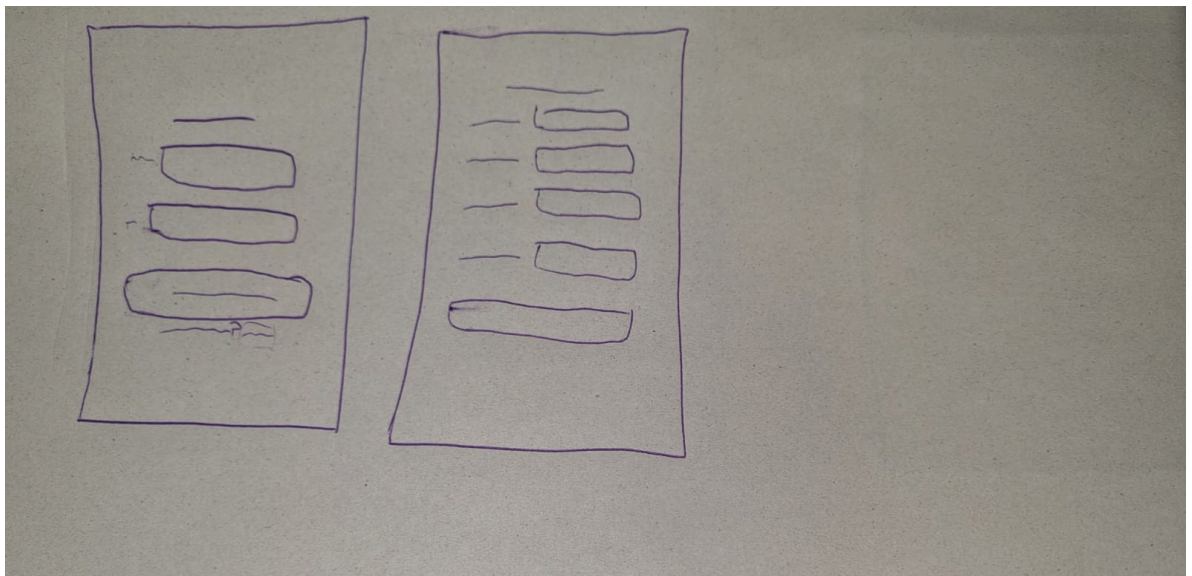
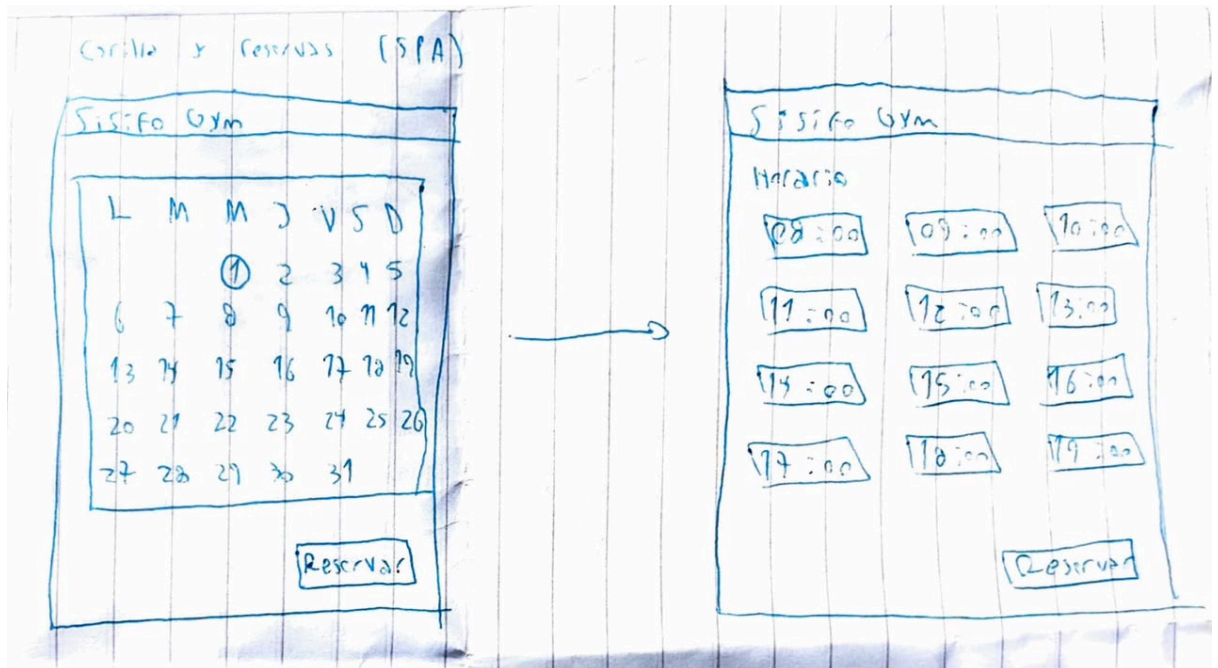
Justificación: Los precios y beneficios de cada plan se actualizan cuando el gimnasio decide modificar tarifas, así que no hay necesidad de actualización en tiempo real.

4. Cuenta de Usuario → SPA

Incluye: Mis Reservas, Mi Pase, Comprar/Renovar pase, Mis Datos.

Que se actualizará: El listado de reservas, el estado del pase, y el formulario de compra/renovación.

Bocetos a mano:



Wireframe digital:

<https://canva.link/7evjzh5cfp1zay7>

Cómo trabajamos

Prompt general: Actúa como un arquitecto de software web. Tengo un proyecto de Sisífo gym. Propón un sitemap (max 4 módulos) para el sitio web y recomiéndame qué secciones deberían ser MPA y cuáles SPA, justificando el impacto en la experiencia de usuario y el rendimiento. Adjunto esta actividad anterior que ayuda a dar más contexto del negocio.

Respuesta obtenida:

Los 4 módulos y su naturaleza

1. Módulo público (MPA) — Home, precios/planes, contacto, landing de "primera clase gratis". Es la puerta de entrada para gente que *todavía no es socia* y llega mayormente desde Google o redes.

- Por qué MPA: cada página necesita indexarse individualmente (SEO), el contenido es mayormente estático, y el usuario entra "en frío" — el costo de un primer TTFB/FCP rápido con HTML ya renderizado pesa más que la fluidez de navegación entre 3-4 páginas que casi no comparte estado.

2. Autenticación (MPA) — Login, registro, recuperar contraseña. Son formularios simples, de bajo tráfico repetido por sesión, sin necesidad de estado compartido con el resto del sitio.

- Por qué MPA: el server-rendering en el login reduce superficie de ataque (menos JS de terceros ejecutando en la página de credenciales) y el "costo" de un refresh de página aquí es irrelevante para la experiencia — el usuario lo hace una vez por sesión.

3. Grilla y reservas (SPA) — Cubre directamente RF-01 (grilla dinámica con skeleton loaders) y RF-02 (indicador de ocupación en tiempo real).

- Por qué SPA: RF-02 exige que el estado de "cupos disponibles" se actualice sin recargar la página mientras el usuario tiene la pantalla activa, y que el botón "Reservar" se bloquee al instante si el cupo se agota. Eso es imposible de lograr limpiamente con

recargas de MPA — necesitás una conexión persistente (WebSocket/polling) y un árbol de componentes vivo en el cliente que reaccione a esos eventos.

4. Panel de usuario (SPA) — Dashboard de reservas, checkout de renovación, cancelación. Cubre RF-03 y RF-04.

- Por qué SPA: RF-04 pide explícitamente **Optimistic UI Update** (el turno desaparece al instante, con rollback si falla) — eso es un patrón de estado en cliente, no algo que un MPA pueda ofrecer sin sentirse "roto". RF-03 pide un checkout de un click con estado **loading**/debounce sin salir de la pantalla, que también requiere que la SPA mantenga el contexto de la transacción sin perder el estado del usuario.

Justificación del split (impacto en UX y performance)

- **SEO y adquisición vs. retención:** Sísifo Gym necesita que gente nueva lo encuentre buscando "gimnasio + clases + [zona]" — eso lo gana el módulo público como MPA, indexable página por página. Una vez que el usuario ya es socio y entra a reservar o pagar, el SEO deja de importar y lo que importa es la fluidez — ahí gana la SPA.
- **Rendimiento percibido:** en el público, MPA gana en *First Contentful Paint* porque no hay que descargar y bootear un framework antes de mostrar contenido. En reservas/dashboard, la SPA gana en *interacciones repetidas* (reservar, cancelar, filtrar la grilla) porque no hay round-trip completo de página por cada acción — solo llamadas a API, que es justo lo que RF-01/RF-02/RF-04 piden.
- **Costo de desarrollo/mantenimiento:** no hace falta que todo el sitio sea una SPA. Un patrón híbrido (ej. Next.js/Astro con SSR/SSG para público+auth, y un "app shell" React/SPA para grilla+dashboard) evita pagar el costo de un bundle JS pesado en las páginas donde nadie interactúa mucho, y lo concentra donde el estado en tiempo real realmente lo justifica.

Si querés, puedo bajar esto a un diagrama de arquitectura técnica (frontend/backend/websockets) para completar la actividad.